إعداد عربي تعليمي · الدفعة الأولى

ما بعد «فايب كودينغ»

من البرمجة بالمحادثة إلى الهندسة المُعانة

استنادًا إلى كتاب

آدي عثماني

Beyond Vibe Coding
From Coder to AI-Era Developer
First edition · O'Reilly Media

مقدمة · الفصل الأول · معجم
مسودة تعليمية جزئية، وليست ترجمة حرفية أو رسمية.
الفصول من الثاني إلى الحادي عشر لم تُعد بعد.

10 أكتوبر 2026 · Markdown + PDF

محتويات هذه الدفعة

مسار تأسيسي بالعربية: اقرأ الفكرة، شاهد المخطط، ثم جرّب ولاحظ.

مقدمة الطبعة العربية التعليمية 3

الفصل الأول: المقدمة — ما «فايب كودينغ»؟ 5

1.1 طيف البرمجة بالذكاء الاصطناعي: من «فايب كودينغ» إلى الهندسة المُعانة 5

1.2 ما بعد أسطر الشيفرة: البرمجة بالقصد 10

1.3 الإنتاجية وإتاحة البرمجة وتغيّر طبيعتها 12

1.4 لمحة عن الأدوات: النظام البيئي الناشئ 14

1.5 نماذج الذكاء الاصطناعي: مشهد توليد الشيفرة 18

1.6 النماذج الكبرى 19

1.7 اختيار النموذج المناسب لاحتياجاتك 21

1.8 فوائد «فايب كودينغ» وحدوده: نظرة متوازنة 21

1.9 الخلاصة والخطوات التالية 27

معجم الدفعة الأولى 28

ملاحظات الإعداد والمراجع 31

خريطة إنجاز الكتاب 33

العناوين روابط داخلية. الأرقام تشير إلى صفحات هذه النسخة العربية، بما فيها الغلاف. إحالات المصدر في المتن تشير إلى صفحات الملف الإنجليزي. يمكنك كذلك التنقل من العلامات المرجعية في قارئ PDF.

مقدمة الطبعة العربية التعليمية

يتغيّر اليومَ طريقةُ بناء البرمجيات تغيّرًا عميقًا. لم يعد المطوّر يكتب الشيفرة سطرًا سطرًا فحسب؛ صار يشارك الذكاءَ الاصطناعي في صناعتها. هذه الطبعة العربية تستند إلى كتاب ما بعد «فايب كودينغ» لآدي عثماني (أورايلي، الطبعة الأولى 2025). نقدم مادته بلغة عربية واضحة للمبتدئ والمتوسط، ونحافظ على ترتيب أفكار الأصل وأقسامه.

لمن هذا الكتاب؟

يخاطب الكتاب ثلاثة أنواع من القراء:

- **المطوّر الخبير وقائد الفريق.** تريد مضاعفة أثرك؟ فوّض العمل المكرر إلى الذكاء، وارفع دورك إلى التصميم ووضع الاستراتيجية، من غير أن تفرّط في الجودة.
- **المهندس صاحب العقلية المنتجية.** ترى الشيفرة وسيلة لا غاية. سيضيّق الكتاب المسافة بين فكرتك وتنفيذها: تجرّب سريعًا، تكرّر، وتشحن منتجًا كان يستغرق شهورًا.
- **مدير الهندسة والمدير التقني.** ترتّب الفرق والتوظيف والجودة في عصرٍ ينتج فيه مهندس واحد ما كان يحتاج فريقًا كاملًا.

أما المفارقة التي يسجلها المؤلف فهي أن الذكاء ينفع الخبير أكثر مما ينفع المبتدئ. السبب بسيط: الذكاء أشبه بمطوّر مبتدئ شديد الحماس. يكتب بسرعة، لكنه يحتاج من يراجعه ويصححه كلما قلت خبرته. لذلك يسمي المؤلف هذه الظاهرة «مفارقة المعرفة»: الخبير يسرّع ما يتقنه أصلًا، بينما يحاول المبتدئ أن يتعلم بالأداة نفسها، فتختلف النتيجة تمامًا.

ومن هنا لا يعد هذا الكتاب مدخلًا إلى البرمجة لمن لا يعرفها إطلاقًا. فالحكم على الشيفرة — أصحيحة هي أم هشة؟ — يحتاج خبرة سابقة.

إضافة تعليمية: إن كنت جديدًا على البرمجة، فاقرأ هذا الكتاب خريطةً لمهارة جديدة، وتعلّم أساسيات البرمجة من مصدر تأسيسي موازٍ. فالكتاب يفترض أنك تميز الشيفرة السليمة من الهشة حين تراها.

ماذا نتوقع من الكتاب؟

يتتبع الكتاب تغيّر دور المطوّر: من كتابة التعليمات إلى توجيه الذكاء نحو الهدف. ويبقى الإنسان مصدرَ الإبداع وفهمِ المستخدم؛ الذكاء يضخّم عملك ولا يستبدلك، إن أحسنت استعماله.

- **الجزء الأول (الأساسات).** يبيّن أين يتألق «فايب كودينغ»: منتج جديد، نموذج أولي، تطبيق نمطي، أو شيفرة تكامل. ويبين أين يلزم الحذر: الخوارزمية المعقدة، والمستوى المنخفض، والمسألة غير المسبوقة. ويبقى الإنسان هو الحَكَم على الجودة.
- **الجزء الثاني (التطبيق العملي).** في سير العمل: متى تعيد التوليد بدل التنقيح، وكيف توجه الذكاء من عدة زوايا، وكيف توازن السرعة مع النظافة حتى لا تتراكم فوضى.
- **الجزء الثالث (الثقة والاستقلالية).** في الأمان والموثوقية والأخلاقيات والأدوات: بيئات مثل Cursor وWindsurf تدمج نماذج Anthropic وGoogle وOpenAI وتفهم قاعدة شيفرتك كلها.

وينظر المؤلف إلى الأمام: طرق بناء أكثر تجريدًا، واعتماد أقل على المكتبات العامة، وبرمجيات تطوّر نفسها بحلقات تغذية راجعة. وفي ذلك المستقبل تعلو قيمة الإبداع وفهم المستخدمين، لأن حاجز كتابة الشيفرة سينخفض كثيرًا.

إضافة تعليمية: تجد خريطة الفصول بالعربية وحالة كل فصل في «خريطة الإنجاز» آخر هذه الدفعة. وهي مشتقة من `manifest.json` في مشروع الكتاب. وذِكْرُ فصل في الخريطة لا يعني أن محتواه أُعدّ.

كيف تقرأ هذه الطبعة؟

1. اقرأ الفكرة بالعربية، ثم انظر إلى الشكل المعاد رسمه بتسميات عربية.
2. جرّب ما يمكن تجريبه بأمان. أمثلة هذا الفصل مفهومية في أغلبها، وما يُنفذ منها موسوم ومفحوص.
3. ارجع إلى المعجم عند أول ظهور لكل مصطلح، والتزم المقابل نفسه.
4. ميّز مادة المؤلف من إضافاتنا: وسما **إضافة تعليمية** و**توضيح حديث** يفصلان بينهما.

اتفاقيات هذه الطبعة

- أسماء الأدوات والنماذج تبقى باللاتينية: `VSCode`، `Cursor`، `Claude`.
- نشرح المصطلح عند أول ظهور، ثم نلتزم مقابله الواحد (راجع المعجم).
- إحالاتنا إلى الأصل تشير إلى صفحات ملف PDF الإنجليزي المرفق، بترقيم فيزيائي يبدأ من 1.
- هذه الطبعة إعداد تعليمي جزئي يُراجع دفعةً دفعة، وليست ترجمة رسمية ولا نسخة كاملة.

حقوق الكتاب الأصلي لأصحابها، ووجود هذا الإعداد لا يمنح إذنًا بنشر الأصل أو توزيعه.

الفصل الأول: المقدمة — ما «فايب كودينغ»؟

يعيد الذكاء الاصطناعي تشكيل طريقة بنائنا للبرمجيات. تخيّل أنك تصف ما تريد بكلمات عادية، ويحوّل الذكاء وصفك إلى شيفرة تعمل. هذا هو «فايب كودينغ»: تبدأ بالوصف، وتدع نموذجًا لغويًا كبيرًا (LLM) يكتب التفاصيل. صاغ المصطلحَ أندريه كارباثي ليصف طريقة جديدة يكتفي فيها المطوّر بتوجيه الذكاء بدل كتابة كل سطر.

سيغوص المؤلف في معنى هذا النهج للمطوّر المحترف، ويقارنه بما يسميه «الهندسة المُعانة بالذكاء الاصطناعي»، أي إدخال الذكاء في كل مرحلة بخطة وقيود واضحة. وسيرى كيف يتغير دور المطوّر، وأي الأدوات أنفع، ومتى يتألق الذكاء ومتى يتعثر.

في هذا الفصل نتابع تحوّل المطوّر من كتابة تعليمات مفصلة إلى التعبير عن قصده وترك الذكاء ينفّذ (انظر الشكل 1-1).



الشكل 1-1 — البرمجة بالقصد: يحدد المطوّر الهدف، ويحوّله الذكاء إلى شيفرة. رسم تعليمي مستند إلى فكرة الشكل في الأصل.

1.1 طيف البرمجة بالذكاء الاصطناعي: من «فايب كودينغ» إلى الهندسة المُعانة

لاحظ المؤلف انقسامًا بين المطوّرين في طريقة استعمال الذكاء. في طرفٍ «فايب كودينغ»: وصف حرّ واستكشاف سريع. وفي الطرف الآخر «الهندسة المُعانة»: إدخال الذكاء في كل مرحلة بخطة وقيود. النهجان يستعملان ذكاءً قويًا، لكن الهدف والجمهور والتوقعات تختلف.

1.1.1 نهج «فايب كودينغ»: البرمجة بالمحادثة

في هذا النهج تتخذ النموذجَ شريكًا يكتب عنك الشيفرة، وتتفرغ أنت للهدف الأكبر. تحاوره بلغة طبيعية: تصف الميزة، تراجع اقتراحه، وتكرّر. هكذا تنقلب المعادلة التقليدية التي تتطلب تخطيطًا دقيقًا وتنقيحًا شاقًا. ويلخص توصيفٌ ورد في تدوينة كارباثي هذه الروح: «ليس هذا ترميزًا حقًا؛ أرى أشياء، وأقول أشياء، وأشغّلها، وألصقها، وهي تعمل غالبًا». وكما يقول جنسن هوانغ، الرئيس التنفيذي لإنفيديا، صارت «لغة البرمجة الجديدة الأشهر» بفضل الذكاء هي الإنجليزية، لا جافا ولا بايثون.

يصبح المطوّر موجّه نتائج بدل كاتب تعليمات. ويروي كارباثي أنه يبني تطبيق ويب بقبول اقتراحات النموذج كلها: «أقبل الكل دائمًا، ولم أعد أقرأ فروق التغييرات... وإن ظهرت رسالة خطأ ألصقها كما هي... وأحيانًا يعجز النموذج عن إصلاح العلّة، فأتجاوزها أو أطلب تغييرات عشوائية حتى تزول». تنمو الشيفرة أبعد مما يكتبه عادة، لكن المشروع يكتمل سريعًا. الهدف: سرعة واستكشاف بأقل احتكاك.

تلاقَت ثلاثة اتجاهات فجعلت هذا ممكنًا. أولًا، صارت مساعدات الترميز جيدة جدًا في توليد الشيفرة وتصحيحها بعدما تدربت على كم هائل من شيفرة GitHub. ثانيًا، ظهرت أدوات تدمج النماذج في سير العمل بسلاسة. ثالثًا، صار المجتمع يأتمن الذكاء على قطع أكبر من العمل: لم يعد إكمالًا تلقائيًا، بل تسليم دوال وملفات كاملة. عمليًا، تشعر أن لديك عددًا لا ينتهي من مبتدئين متحمسين ينفذون طلباتك بسرعة الحوسبة السحابية.

وأبرز الوعود قفزة الإنتاجية. يذكر روّاد أنهم يبنون ميزات أسرع بعشر إلى مئة مرة. تولّد الشيفرة النمطية في ثوانٍ، وتصلح العلل في لحظات، وتكتب الاختبارات والتوثيق. فيتقلص عمل أيام إلى ساعات. وقد يبني مطوّر واحد في عطلة نهاية أسبوع تطبيقًا كاملًا كان يحتاج فريقًا أسابيع. وليست ضجة؛ فاستطلاعات يذكرها المؤلف تفيد أن 75% من المطوّرين دمجوا الذكاء في عملهم، وأن شركات سجلت تحسنًا كبيرًا في سرعة التطوير.

مثال ملموس: يريد مطوّر تطبيقًا يعدّ كلمات نصّ ويقدّر زمن قراءته. يفتح بيئة معززة بالذكاء ويصف فكرته، فيخرج نموذج أولي خلال دقائق. ثم يقول: «اجعل العدادات ملونة وأضف تصدير PDF»، فيحدّث الذكاء الشيفرة. النتيجة أداة تعمل وتنشر بنقرة، في أقل من عشر دقائق. وحتى غير المهندسين دخلوا الميدان: يروي الأصل أن مسوّقًا بلا خلفية برمجية بنى بمساعد ذكي مئة أداة ويب بلغت صدارة Product Hunt. حين ينخفض حاجز صنع البرمجيات، يتسع من يستطيع التطوير أصلًا.

لكن للنهج محاذير. إذا فوّضت الكثير للذكاء، فقد تحصل على شيفرة «تعمل» في الحالة السهلة وتخفي عللًا وقرارات سيئة. بلا خطة أو قيود قد يفتقر الناتج إلى معالجة الأخطاء والفحص الأمني وقابلية التوسع. اطلب «نظام دخول مستخدمين»، فيخرج مسار يعمل سريعًا لكنه ربما يستعمل تشفيرًا مبسطًا أو مكتبة فيها ثغرة؛ نشرُه بلا فحص مجازفة. وينقل المؤلف عن خبير: «الوصولُ بأسلوب فايب كودينغ إلى قاعدة إنتاج مجازفةٌ واضحة؛ أغلب عملنا تطوير أنظمة قائمة تعتمد على جودة الشيفرة ووضوحها».

ومشكلته الثانية تهميش التخطيط. الهندسة التقليدية تقدر تصميم الوضوح: نموذج البيانات، والأنماط الملائمة، ومواصفة موجزة. أما «فايب كودينغ» فيبدأ بلا سقالة ويغوص في التنفيذ. وقد تجد نفسك في زاوية: اختار الذكاء مكتبة أو نهجًا لم تقصده، فعليك توجيهه للعودة أو التعايش معه. بلا مخطط أولي قد تخرج المعمارية مرتجلة؛ وهذا مقبول لنموذج سريع، ومقلق لقاعدة شيفرة كبيرة يهم فيها الاتساق.

ليس النهج سيئًا في ذاته؛ ظهوره امتداد لدمقرطة البرمجة، كما فعلت منصات Low-Code ولغات السكربت. وقد يبني غير المهندس تطبيقًا بسيطًا بالوصف وحده. وللمطوّر الخبير هو أداة عصف قوية: pseudocode بنتائج فورية قابلة للتشغيل. المفتاح معرفة الحدود: سرعة بلا انضباط تنتج برمجيات هشة، لذا يلزم إنسان متيقظ في الحلقة. يذكّر المؤلف نفسه: «فايب كودينغ ليس عذرًا لعمل رديء». هو بداية الحل لا نهايته.

1.1.2 نهج الهندسة المُعانة: هيكل واضح مع شريك ذكاء اصطناعي

في الطرف المقابل تقع «الهندسة المُعانة»: بناء منظم يبقى فيه المطوّر صاحب القرار، ويشارك الذكاء في كل خطوة. يشمل ذلك إكمالًا تلقائيًا، ومحادثة، وهجرة شيفرة، وكشف علل، وتوليد اختبارات، وتوليد شيفرة جزئيًا أو كاملًا (انظر الشكل 2-1).

تبدأ بخطة ولو موجزة: ماذا ستبني؟ وما القيود ومعايير القبول؟ ثم تدخل الذكاء لتسريع أجزاء من الخطة. مقابل «التوجيه أولًا» في فايب كودينغ، نسمي هذا «الخطة أولًا». قد تكون الخطة وثيقة متطلبات مصغرة أو قائمة مهام. الفرق الجوهري: ترسّخ القصد والقيود قبل أن تطلق الذكاء.



الشكل 2-1 — سير «الخطة أولًا»: مواصفة، فتوجيهات، فمراجعة المقتطفات، فدمج الحلول المقبولة. رسم تعليمي مستند إلى فكرة الشكل في الأصل.

تأمل مطوّر React كُلّف مكوّن لوحة تحكم. يبدأ بكتابة مسؤولياته وواجهته:

```
Dashboard component shows a list of analytics cards,
supports filtering by date range,
and has refresh and export buttons. It should fetch data
from our API (with proper
error handling), and must follow our design system for
styling.
```

هذه الخلاصة مواصفة. قد يرسم نموذج بيانات سريعًا أو يحدد دوال موجودة لإعادة استعمالها. عندها فقط يدخل الذكاء ليولّد هيكل المكوّن من الوصف. ولأن الطلب واضح، يرجّح أن يتوافق الناتج مع حاجة المشروع. الشيفرة ليست مفاجأة؛ إنها نتاج طلب جيد الصياغة.

ولا تتوقف الهندسة المُعانة عند مكوّن واحد؛ تتخلل دورة الحياة كلها بضبط. في المهام الروتينية يقترح الإكمال التلقائي الأسطر التالية وأنت تكتب. وبعد استقرار ميزة تستعمل الذكاء لتوليد حالات اختبار: تطعمه المواصفة فيقترح حالات حدّية تفحصها. الفكرة تعزيز المهندس لا استبداله: تفكر أنت في المنطق وتتحقق من الصحة، ويحمل الذكاء عنك العمل المكرر.

وفي هجرة الشيفرة يفيد الذكاء كثيرًا. حوّل مكوّن React من صنف إلى دالة بخطافات: بدل العمل اليدوي كله، اطلب من المساعد التحويل أو رسم الخطوات. وبفهمك النمطين تراجع المسودة وتهذبها. هذا استخدام منظم يعالج مهمة محددة — «انقل هذا من Redux إلى React Context API» — لا تفويضًا مفتوحًا.

ولعل أوضح صوره توليد تطبيق مصغر من مواصفة مفصلة: تصف تطبيق قائمة مهام بواجهة React وخادم Node.js مع تحقق وتحديثات لحظية، فتبني الأداة السقالة والمكوّنات ومخطط قاعدة البيانات. ليس هذا سحرًا؛ إنه نسخة مسرعة مما يفعله مهندس مجتهد عند بدء مشروع. والمهم أن إبداع الذكاء مقيّد بقيود المواصفة. والمطوّر الخبير يعامل الناتج مسودة أولى لا جاهزًا للإنتاج: يشغّله، ويكتب اختبارات، ويراجع الشيفرة، ويهذب. يطبق صرامته المعتادة، مسرّعة بقدرة الذكاء على إنتاج شيفرة كثيرة من مخطط.

أهداف الهندسة المُعانة مختلفة: لا مجرد شيفرة تعمل بسرعة، بل شيفرة جيدة بكفاءة أعلى. فريق يطبق النهج قد يقول: «نسلّم الميزة أسرع مرتين، بلا تنازل عن معاييرنا». وجمهوره مطورون محترفون وفرق لها عمليات راسخة — مراجعة واختبار وخطوط نشر — لا تريد التخلي عنها. والتوقع أن يبقى الإنسان صاحب القرار، وأن تخضع كل قطعة مولدة للتدقيق نفسه كأن مبتدئًا كتبها. عامل الذكاء متدربًا لديك لا بديلًا عنك؛ فكما لا تنشر شيفرة متدرب بشري بلا مراجعة، لا تنشر شيفرة ذكاء بلا فهم.

1.1.3 عقليتان مختلفتان وتوقعات مختلفة

النهجان عقليتان مختلفتان. «فايب كودينغ» استكشافي من أعلى إلى أسفل: تبدأ بفكرة واسعة وتدع التنفيذ ينبثق من الحوار مع الذكاء — كارتجال الجاز: بنية قليلة وحرية كثيرة. والهندسة المُعانة منهجية تكرارية: كتأليف كلاسيكي تبدأ بموضوع (متطلباتك) وتطوره بانضباط، مع لمسات ارتجال داخل حدود النوتة. كلاهما ينتج «موسيقى»، لكن العملية والنتيجة تختلفان.

إن مارست «فايب كودينغ» فتوقّع المفاجآت: قد يأتي الذكاء بنهج لم تكتبه أنت — مكتبة أخرى أو أسلوب أقل ألفة. جزء من الجاذبية التعلم من هذه المفاجآت أو تجاوز ما يُمَلّ. لكن توقّع العثرات أيضًا؛ المسافة الأخيرة الصعبة مسؤوليتك. السحر حقيقي لكنه ليس تامًا.

وإن مارست الهندسة المُعانة فتوقعاتك أهدأ وأصدق للمشاريع الطويلة: الذكاء يوفر وقتًا ويلهم حلًا أو اثنين، لكنه لا يقوم بعملك كله. وقد يستعمل المهندس الجيد توجيهًا حرًا بجرعات صغيرة داخل إطار أكبر: أثناء تنفيذ وحدة محددة يسأل «ولّد دالة سريعة لتنسيق هذه التواريخ»، ثم يعود لوضع المهندس فيدمجها ويفحصها. العقلية: الذكاء متعاون يعمل بتوجيهك؛ تخصّص له ما يبرع فيه — شيفرة نمطية ومتكررة — وتتولى أنت المنطق الحرج والتكامل والمراجعة النهائية.

وتشمل التوقعات هنا إنتاجية أعلى وأخطاء روتينية أقل وفضاء حلول أوسع. لكنك تستثمر وقتًا في التحقق: تنقيح شيفرة كتبها الذكاء ما يزال تنقيحًا، تشغّل اختبارات وتتبع الشيفرة عند الحاجة. الفرق أنك قد تنقح شيفرة كتبها غيرك — تجربة جديدة لها منحنى تعلم (يفصّلها الفصل الخامس).

وتكشف الأهداف الفرق الجوهري: «فايب كودينغ» يحسّن للسرعة القريبة، والهندسة المُعانة لسرعة مستدامة وموثوقية. يقول الأول: «أريد تشغيل التطبيق الليلة لأرى إن نجحت الفكرة». ويقول الثاني: «أريد الميزة سريعًا، لكن متينة لتعيش في قاعدتنا سنوات». الأول يكتفي بأن تعمل الشيفرة؛ والثاني يهتم بأن تكون نظيفة ليبني عليها غيره. ولكل نهج جمهوره: الأقل خبرة يميل إلى فايب كودينغ لانخفاض حاجزه واشباعه الفوري؛ والمخضرمون يفضّلون الهندسة المُعانة لأنهم اكتووا بالشيفرة الهشة، فيبدؤون بـ«لنفعلها صح ولو بأدوات أسرع».

1.1.4 أين موقعك على الطيف؟

من المغري أن نسأل: أيهما أفضل؟ الحقيقة أنهما ليسا فئتين متنافيتين؛ هما طرفا طيف، وتمزج سير العمل الواقعية بينهما. قد يبدأ مطوّر مشروعًا باندفاعة فايب كودينغ لسقالة جديدة، ثم يتحول إلى وضع المهندس لتثبيته. أو يلتزم الانضباط عمومًا لكنه — لسكربت عابر — يقول: «سأجرب فايب كودينغ وأرى». المفتاح فهم المقايضات واستعمال النهج المناسب في موضعه.

عدّ «فايب كودينغ» مركبة استكشاف سريعة: تشق طريقًا غير مطروقة وتصلح للاكتشاف. والهندسة المُعانة قطار موثوق على قضيبين: تمدّ السكة أولًا، لكنه أأمن لبلوغ وجهة محددة. وعلى المطوّر المتوسط والمتقدم قيادة المركبتين، ويختار بحسب المهمة. للابتكار السريع — هاكاثون أو التحقق من فكرة — يمنحك فايب كودينغ الزخم، وتذكّر أن تشدّ الأشياء إن أعدت استعمال الشيفرة. ولميزة منتج قابلة للصيانة، يضمن الميل إلى الهندسة المُعانة ألا تستقر في قاعدتك كتلة سوداء لا يفهمها أحد.

ولاحظ المؤلف أن المطوّرين كلما ازدادوا خبرة بالأدوات، مال استعمالهم من طرف الانطباع إلى طرف الهندسة. في البداية يسحرهم توليد كتل كاملة من توجيه واحد. وبعد شهر العسل يحلّ الواقع: يرون أين يتألق الذكاء وأين يتعثر، فيتعلمون تجزئة المشاكل وإطعامها قطعًا بدل طلب الحل كله. ينتقلون من «فنان توجيه» إلى «قائد أوركسترا» يوجّه الذكاء بيد خبيرة. وفي ممارسته صار المؤلف أعمد: يكتب قطعًا صغيرة من pseudocode أو تعليقات ويطلب من الذكاء إكمالها، فيحوز سيولة فايب داخل هيكل يسيطر عليه.

ومن الجدير ذكره أن الأدوات تتطور لتخدم الطيف كله. في طرف واجهات محادثة وبيئات بلغة طبيعية مصممة لفايب كودينغ قد لا ترى فيها الشيفرة حتى تطلبها. وفي الطرف الآخر تضيف بيئات التطوير ميزات تندمج بالترميز التقليدي: منظّمات ذكية، ومولّدات توثيق، وبوتات تنشئ طلبات دمج وتقترح تغييرات. هذه الأدوات تشجع عقلية هندسية لأنها تدخل سير العمل المعتاد — تحرير ومراجعة واختبار — مع استفادة من الذكاء. وقد يبهت التمييز مع الزمن: ما نشعر به اليوم «فايب» سيكسب حواجز حماية، وما نشعر به «هندسة» سيصير أكثر سيولة. والمثال الأعلى مستقبل نتنقل فيه على الطيف بحرية: نستكشف حين نريد، ونشدّ الزمام بممارسات رصينة حين يحين وقت التصلب والشحن.

1.2 ما بعد أسطر الشيفرة: البرمجة بالقصد

لعقود عنى الترميز كتابة تعليمات: سطرًا سطرًا يخبر الحاسوب كيف يفعل. كل دالة وحلقة وشرط يصوغها إنسان بعناية. «البرمجة بالقصد» تقلب المعادلة: يركز المطوّر على الهدف — ماذا أريد للبرنامج أن ينجز؟ — ويعبّر عنه بلغة عالية المستوى، ويدع الذكاء يستنتج الشيفرة.

فكّر هكذا: الترميز التقليدي كإعطاء اتجاهات خطوة بخطوة، والترميز بالقصد كإخبار السائق بوجهتك وترك اختيار الطريق له. بالتركيز على «ماذا» بدل «كيف» يعمل المطوّر على مستوى تجريد أعلى. الفكرة ليست جديدة — البرمجة المرئية ومنصات Low-Code وعدت برفع التجريد — لكن تقدم الذكاء جعل وصف سلوك معقد والحصول على شيفرة تعمل أمرًا عمليًا أخيرًا.

1.2.1 صعود التوجيه (البرومبت): من التعليمات إلى الأوصاف

في قلب هذا التحول يقف التوجيه (البرومبت): ما تعطيه لنظام ترميز ذكي. هو وصف لما تريد، لا تعليمات لكيفية فعله. بدل كتابة حلقة لتحليل ملف، قد تكتب:

```
Read this CSV file and extract the email addresses of all
users older than 18.
```

فيحاول الذكاء توليد شيفرة تحقق الوصف.

لماذا الآن؟ لأن النماذج اللغوية الكبيرة تقدمت سريعًا في فهم النصوص وتوليدها، ومنها لغات البرمجة. تدربت على كم هائل من الشيفرة والنص، فصارت تقرأ وصفًا لسلوك برمجية وترجمته إلى شيفرة تنفذه. تعلمت أنماط وصف البشر للمهام، وكيف تترجم تلك المهام إلى شيفرة. ويعني هذا أنك تكتب أوصاف الميزات بلغة طبيعية أو pseudocode، ويتولى الذكاء الكتابة الصحيحة نحويًا. صار التوجيه وحدة تفكيرك الجديدة: تعبير مكثف عن القصد. انتقلنا من «افعل كذا ثم كذا» إلى «أريد إنجاز كذا»، ثقةً بأن الذكاء يملأ الفراغ.

ومن المهم ملاحظة أن كتابة توجيه جيد مهارة في حد ذاتها (يفصّلها الفصل الثالث). التوجيه الغامض يقود إلى شيفرة خاطئة أو غير كفؤة، كما تربك المتطلبات الغامضة مبرمجًا بشريًا. وكلما أوضحت قصدك، طابق الناتج حاجتك. لذا يسمي كثيرون كتابة التوجيه «محوارية البرمجة الجديدة».

1.2.2 كيف يعمل: الدورة التكرارية ودور الذكاء الاصطناعي في توليد الشيفرة

فكيف ينتقل الذكاء من وصفك الحر إلى شيفرة تعمل؟ يكمن «السحر» في قدرة النموذج على فهم السياق وتوليد النص. «الكبير» في النموذج اللغوي يشير إلى عدد البارامترات — مليارات فأكثر — تمكنه من التقاط تعقيد اللغات. تدرب على مستودعات شيفرة ومنتديات وتوثيق، فتعلم النحو ودلالات حل المشكلات. حين تتفاعل معه تستفيد من هذه المعرفة الواسعة. ببساطة:

فهم التوجيه. يحلل النموذج نص توجيهك («ولّد دالة تفحص إن كان عدد أوليًا»). وقد تدرب على أمثلة لا تحصى، فيستعمل أنماطًا إحصائية لاستنتاج طلبك، ويتوقع الإكمال الأرجح بشيفرة معقولة.

استثمار السياق. يأخذ النموذج سياقًا إضافيًا وراء التوجيه: في بيئة تطوير بمساعد ذكي يراعي محتوى الملف وأسلوبك وتعليقاتك وملفات ذات صلة. هذا السياق يساعده على توليد شيفرة تلائم مشروعك، كما يقرأ مطوّر بشري الشيفرة المحيطة ليفهم الخطوة التالية.

توليد الشيفرة. بعد فهم قصدك (أو تخمينه) يولّد الشيفرة رمزًا رمزًا، باحتمالات تعلمها من التدريب. النموذج لا «يفكر» كما نفكر؛ لا مصرف لديه يفحص الشيفرة. إنه بارع في متابعة النص بما يرجح أنه شيفرة صحيحة لأنه رأى أمثلة كثيرة. وإن وضَح التوجيه والسياق، جاء الناتج دقيقًا وملتزمًا بأفضل ممارسات رآها.

التحقق بإشراف بشري. لا يركّب الذكاء تطبيقك وينشره عنك؛ أنت في الحلقة. تراجع الشيفرة وتختبرها وتقبلها أو تعدلها، وقد يشرحها إن سُئل. دوره مساعد يسوّد الشيفرة، وأنت صاحب القرار الضامن لصحتها وملاءمتها.

وما يثير الإعجاب أن هذا يحدث في ثوانٍ. يدخل وصفك إلى محرك توقع فيخرج شيفرة مرجحة؛ يكاد يشبه تعاونًا مع خبير يستحضر فورًا كيفية تنفيذ أي شيء. وأهم ما ينبغي فهمه أن «فايب كودينغ» عملية تكرارية تعاونية: لن تكتب توجيهًا مثاليًا ويجلس الذكاء يكتب برنامجًا كاملًا بلا عيب؛ بل تتحاوران في حلقة تغذية راجعة تنقل الفكرة الغامضة تدريجيًا إلى شيفرة مصقولة. هكذا قد تبدو دورة نموذجية:

الخطوة	ما يحدث
1. تصف ما تريد	«ولّد دالة تحسب قسطًا شهريًا لقرض بمعطيات الأصل والفائدة والمدة»
2. يقدم الذكاء حلًا أوليًا	شيفرة الدالة بمعاملاتها وصيغتها، وربما تعليقات تشرحها
3. تراجع وتختبر	هل تعالج الحالات الحدّية؟ جرّب: ماذا لو كانت الفائدة 0%؟
4. تهذب طلبك أو شيفرتك	«عدّل الدالة لتعالج فائدة 0% برشاقة»، أو اشرح جزءًا غامضًا
5. يهذب الذكاء الحل	يأخذ ملاحظتك ويضبط الشيفرة فتفحص الصفر وتعالجه
6. كرر حسب الحاجة	اطلب مثلًا توليد اختبارات وحدة، وشغّلها لتتأكد

يشبه هذا البرمجة الزوجية: إنسان يحدد الاتجاه ويعرف المتطلبات، ومساعد ذكي يقترح ويكتب النمطي ويسرّع الممل. ولا يكفي أحدهما وحده للمهام المعقدة: يعتمد الذكاء على الإنسان في التوجيه والتحقق، ويفوّض الإنسان عملًا للذكاء ليتحرك أسرع. والأهم أن التكرار ليس إصلاح أخطاء فقط بل تطوير للحل: تبدأ بتوجيه خشن وتصقل قصدك كلما رأيت ناتج الذكاء. وهذا يشجع التجريب؛ إن أخطأت المحاولة الأولى فلم تخسر كثيرًا — صقل التوجيه وجرّب من جديد. في الترميز التقليدي كتابةُ وحدة ثم رميها محبطة؛ أما مع شيفرة مولدة فكلفة البداية الخاطئة منخفضة، فتستكشف نهجًا مختلفة بحرّية.

1.3 الإنتاجية وإتاحة البرمجة وتغيّر طبيعتها

لماذا البرمجة بالقصد شأن كبير؟ لهذا التحول آثار عميقة:

إنتاجية أعلى. الفائدة الأظهر هي السرعة: ينجز المطورون أسرع حين يتولى الذكاء العمل المكرر. شيفرة روتينية تستغرق ساعات يدويًا — نماذج قواعد بيانات، نقاط نهاية API، سكربتات تنظيف — تُولد في دقائق. دراسات مبكرة تؤكد: أتمّ مطورون استعملوا GitHub Copilot مهمة أسرع بكثير (وجدت إحدى الدراسات خفض زمن بنسبة 55%). وإن ضربت هذه المكاسب في مشروع كامل، لاحت دورات تطوير أقصر بكثير.

إبقاء المطوّر في حالة التدفق. وراء السرعة فائدة نفسية: كتابة النمطي والبحث عن نحوٍ يكسران تدفق المطوّر. ومع ذكاء يحمل هذه المقاطعات يبقى مركّزًا على المشكلة. يبلغ مستخدمون كثر أنهم أقل إحباطًا وأكثر تركيزًا على الجانب الإبداعي — برمجة أمتع قد تحسّن جودة العمل أيضًا.

حاجز دخول أدنى. تطلبت البرمجة تقليديًا تعلم نحو صارم وغرائب مكتبات. مع البرمجة بالقصد ينتقل جزء من هذا العبء إلى الذكاء: قد لا يتذكر القادم نحو فتح ملف، لكنه إن وصف ما يريد ملأ الذكاء التفاصيل. لا يعني هذا أن أي أحد يبرمج أنظمة معقدة بلا معرفة — ما زلت تحتاج فهم ما ينبغي للبرنامج فعله — لكن المنحدر إلى نتائج مفيدة أقصر. وقد يكتب خبير مجال — أحيائي أو اقتصادي — نموذجًا أوليًا في حقله بوصف حاجاته، وإن لم يكن مطورًا محترفًا.

تغيّر الأدوار والمهارات. كلما تولى الذكاء توليد الشيفرة، علت أهمية التصميم المعماري وتفكيك المشكلات والتحقق. وقد تقضي وقتًا أطول في تقرير ما تبني ومراجعة لماذا تعمل الشيفرة، بدل طباعة النحو. قد يتحول «أعرف البرمجة» إلى «أعرف كيف أجعل الذكاء يبرمج». وهذا يدمقرط جوانب ويرفع المستوى الذي يعمل عنده المحترفون، وستظهر أفضل ممارسات جديدة حول توجيه الذكاء (يقدمها الفصل الثالث ويعود إليها الكتاب مرارًا).

الإنتاجية في مقابل الإبداع. وكلما تولى الذكاء الترميز الروتيني، تفرغ البشر لمهام إبداعية أعلى: تحسين تجربة المستخدم، وعصف ميزات جديدة، وحل مسائل خوارزمية صعبة. في هذا السيناريو يزيد الذكاء إنتاجية الثمانين بالمئة المتكررة، ويحرر طاقتك للعشرين بالمئة الابتكارية.

غير أن الصورة ليست وردية كلها؛ ثمة تحديات:

الثقة والصحة. أتأتمن شيفرة كتبها الذكاء؟ إن لم تر كل سطر فقد تمر أخطاء. على المطور اختبار الشيفرة المولدة ومراجعتها بدقة، وعبء ضمان صحتها وأمانها وكفاءته على الإنسان. الثقة العمياء مجازفة.

فقدان مهارات منخفضة المستوى. إن اعتمدت على الذكاء في الروتيني، أفقد تدريجيًا قدرتك على كتابته من الصفر أو تنقيح علل عميقة؟ القلق يشبه إضعاف الآلة الحاسبة لمهارات الحساب. يلزم توازن واعٍ بين الراحة والحفاظ على الأساسيات.

تحوّل مشهد الوظائف. قد تقدر الصناعة مهارات مختلفة: طلب أقل على من يجيد ضخ المنطق النمطي، وأكثر على من يصمم الأنظمة ويدمج المكونات ويتحقق من الصحة.

وفوق ذلك، من أهم عوامل «فايب كودينغ» حجم نافذة السياق. يوفر Gemini أطول نافذة بين النماذج، وقد يقلب المعادلة في المشاريع الكبيرة؛ بعض النماذج يدعم نوافذ تتجاوز مليون رمز تحيط بتطبيق كامل، فيستطيع المطور إطعام قواعد شيفرة كاملة للذكاء ليفهمها. وسيفصّل الكتاب هذه المقايضات آخر الفصل؛ لنتعرف أولًا على الأدوات الناشئة التي تتيح هذه الطريقة.

1.4 لمحة عن الأدوات: النظام البيئي الناشئ

«فايب كودينغ» فلسفة، لكنه يمكَّن بجيل جديد من الأدوات. على المطوّر الخبير أن يتعرف على منصات ونماذج رئيسة تجعل الترميز المُعان فعالًا. جولتنا تشمل VSCode بنظامه البيئي المتنامي، وبيئات مثل Cursor وWindsurf، ونماذج مثل Claude وChatGPT. لا تغطي الجولة وكلاء الترميز في الخلفية؛ يفصّلهم الفصل العاشر. وأنت تقرأ، لا تشغل بحفظ الأسماء؛ المشهد يتطور بسرعة — الهدف فهم أنواع الحلول المتاحة.

1.4.1 VSCode مع Copilot: منصة التطوير الذكية المتكاملة

تحول VSCode من أشهر محرر شيفرة إلى منصة تطوير معززة بالذكاء عبر تكامله العميق مع GitHub Copilot. يريد مايكروسوفت إبقاء قدرات الذكاء داخل البيئة المألوفة التي يستخدمها ملايين المطوّرين يوميًا. يعرض Copilot اقتراحات وشروحًا وتنفيذات آلية استنادًا إلى لغة طبيعية وسياق الشيفرة القائمة؛ وليس إضافة ملحقة بل امتداد طبيعي للمحرر.

تتمحور القدرات حول ثلاثة أنماط. أولًا، إكمال الشيفرة المضمّن: اقتراحات أثناء الكتابة من سطر واحد حتى دوال كاملة؛ يظهر نص شبحي تقبله بـ Tab. ثانيًا، المحادثة: لوحة جانبية تحاور فيها شيفرتك وتطلب تنفيذات. ثالثًا — ولعله الأقوى — وضع الوكيل: يُعطى هدفًا فيختار أدوات وينفذها خطوة خطوة، يحلل قاعدتك، ويقترح تعديلات عبر ملفات، ويشغّل أوامر طرفية، ويستجيب لأخطاء البناء، ويصحح ذاته حتى اكتمال المهمة.

ومما يقوّي تنفيذ Copilot دعمُه بروتوكول سياق النموذج (MCP): طريقة معيارية تكتشف بها النماذج أدوات وبيانات خارجية وتتفاعل معها؛ فيتصل بقواعد بيانات ويستدعي واجهات ويقرأ توثيقًا ويتكامل مع نظامك كله. مثلًا مع خادم GitHub MCP تطلب «أنشئ بلاغًا لكل خلل ناقشناه» فينفذ عبر واجهة GitHub. هذه القابلية للتوسع تحوّله من مولد شيفرة إلى مساعد تطوير شامل يفهم سير عملك كله.

ولاستعماله بفعالية، ابدأ باستكشاف الأنماط حسب تعقيد مهمتك: للإكمالات البسيطة اعتمد الاقتراحات المضمنة وأيقونة الشرارة قرب الأخطاء. ولمهام أعقد انتقل إلى وضع الوكيل بفتح المحادثة واختيار «Agent»؛ وضع مهيأ لتعديلات مستقلة عبر ملفات، ومفيد حين تلزم أدوات وأوامر طرفية. هذه الخلطة من واجهة مألوفة وقدرات متطورة خيار مقنع لفرق تريد ذكاءً بدرجة مؤسسات دون مغادرة بيئتها.

1.4.2 VSCode مع Cline: وكيل ترميز مستقل مفتوح المصدر

قبل استعراض بيئات مبنية لغرض الذكاء، يستحق Cline (المعروف سابقًا بـ Claude Dev) نظرة. يحوّل VSCode إلى بيئة معززة بالذكاء، بفلسفة مختلفة عن Copilot: ليس مساعدًا محكم التكامل، بل وكيل ترميز مستقل يتحمل مهام معقدة متعددة الخطوات من البداية للنهاية. إضافة مفتوحة المصدر تجلب قدرات تتجاوز غالبًا محررات ذكاء احتكارية، مع مرونة VSCode وقابليته للتوسع.

ما يميز Cline نهجه الوكيلي: أعطيته «أنشئ REST API لإدارة المستخدمين مع تحقق»، فلن يولّد شيفرة نمطية فحسب؛ يحلل بنية مشروعك، ويخطط التنفيذ عبر ملفات، وينشئ مجلدات مناسبة، ويثبّت الاعتماديات، وقد يشغّل اختبارات للتحقق. ويعرض كل خطوة — إنشاء ملفات وتعديلات وأوامر — لاعتمادها أو تعديلها. هذا «الإنسان في الحلقة» يوازن الأتمتة والسيطرة.

وتمتد قدراته أبعد من التوليد: أتمتة متصفح لبحث توثيق، وتنقيح علل بتحليل آثار أخطاء عبر ملفات، وتفاعل مع خدمات خارجية عبر MCP. في التنقيح تلصق رسالة خطأ فيتتبع قاعدتك ليحدد السبب ويقترح إصلاحًا وينفذه ويضيف معالجة أخطاء تمنع تكراره. ويعني تكامل MCP اتصاله بقاعدة بياناتك لفهم المخططات قبل توليد استعلامات، وبأدوات إدارة مشاريعك لمواءمة التنفيذ. هذه القابلية للتوسع تحوّله من مولد شيفرة إلى شريك تطوير يفهم نظامك كله.

وللفرق مزايا: بكونه مفتوح المصدر تفحص الفرق شيفرته وتساهم أو تفرّع لحاجات مخصصة — حاسم لأمن وامتثال. ويدعم مزودين متعددين: Claude وOpenAI وGemini وحتى نماذج محلية عبر Ollama، فتمنح الفرق مرونة حسب الأداء والكلفة وإقامة البيانات. ولاستعماله بفعالية: صغ توجيهات مفصلة بسياق المشروع وقيوده، واستفد من تحليله قاعدتك قبل التعديل، ووظف تكراره — بعد تنفيذه ميزة تختبرها وتطلب تهذيبًا في المحادثة نفسها.

1.4.3 Cursor: محرر الشيفرة المدفوع بالذكاء الاصطناعي

من رايات حركة «فايب كودينغ» Cursor، بيئة معززة بالذكاء اكتسبت شعبية سريعة. هو محرر «ذكاء أولًا» (تفريعة من VSCode) يدمج توليد الشيفرة وفهمها في بيئة تطويرك. شعاره «محرر الشيفرة بالذكاء»، وصُمم لتكتب وتعدّل بتعليمات مفهومة: ظلّل دالة واطلب «حسّنها» أو «أضف معالجة أخطاء» فيقترح فورًا. وذكاءه مدرك للمشروع: يفهرس قاعدتك ويفهم سياق ملفاتك، فيقترح أدق من إكمال تلقائي بسيط؛ إنه يحاور شيفرتك كما يحاور ChatGPT لكنه يعرف قاعدتك.

ويعتمد Cursor نماذج متقدمة (غالبًا Claude أو OpenAI حسب إعدادك)، وله شريط محادثة ووضع «Composer» لتوليد متعدد الخطوات. وقد استعمل كارباثي Composer مع نموذج Sonnet حتى حدّث المحرر صوتًا وظهرت الشيفرة فيقبلها أو يصقلها. وتصف تسمية الشكل في الأصل واجهة Cursor بأنها من جيل

جديد من بيئات التطوير: «اترك المحرر يعمل، واذهب لإحضار قهوتك، وعد بميزات مكتملة تعمل». ولا يولّد Cursor الشيفرة فقط بل يعدّل الموجودة حين يُؤمر؛ قد تسأل:

```
Could you make it easier to switch certificates in the
transport listener?
```

فيفهم أنك تشير إلى شيفرتك ويقترح تعديلات في الملف المعني، أو يقرأ ملفات ذات صلة كلف مواصفة (انظر الشكل 3-1). في النسخة المجانية يعرض الفرق في المحادثة لاعتماده؛ وفي الاحترافية يطبق التغييرات تلقائيًا على مساحة عملك.

وَجِّه
صف الميزة أو الإصلاح بلغة طبيعية في المحادثة

يفهرس Cursor مشروعك ويقرأ السياق ذي الصلة

راجِع
مسودة تغييرات تُعرض كفرق (diff) لاعتمادها أو تهذيبها

طبّق، شغّل، والصق أي خطأ ليحلله الذكاء

كرِّر
حلقة: وجِّه، ثم راجِع، ثم اقبل، حتى تكتمل الميزة

الشكل 3-1 — حلقة العمل في محررات ذكية مثل Cursor: فهرسة المشروع والتكرار على التوجيهات حتى ميزات تعمل. رسم تعليمي لا لقطة حرفية.

لاستعماله بفعالية في سير عمل محترف، افتح محادثة وصف الميزة — «أضف نموذج دخول ببريد وكلمة سر مع تحقق ورسائل خطأ» — فيولّد الشيفرة بحالة مسودة؛ تراجعها ثم تضغط «Apply» لدمجها. يتبع كثيرون حلقة: وَجِّه، ثم راجِع، ثم اقبل. وإن لم يكتمل الاقتراح صقل توجيهك («استعمل Tailwind للتنسيق») أو اطلب إصلاح ما ترصده («الآن عالج حالة البريد المسجل مسبقًا»). جوهرًا، تحاور شيفرتك حتى تستقيم.

ويبرع Cursor في فهم الأخطاء: شغّلتَ شيفرتك وجاء أثر خطأ؟ الصقه في المحادثة وغالبًا يحلله الذكاء ويقترح إصلاحًا — يتحول التنقيح إلى تعاون بدل بحث يدوي. مع ذلك تحقق؛ فقد لا يصيب من أول مرة. ونصيحة أخرى: استفد من مراعاته ملفات متعددة — اختر مجموعة ملفات أو اذكر سياق المشروع فيراعي قاعدتك كلها: «أضف نقطة نهاية API تدعم نموذج الدخول واربطها بالواجهة». هذا السياق الشامل نقله مقارنة بمساعدين

يعملون ملفًا ملفًا. وخلاصة: Cursor كمبرمج زوجي ذكي داخل بيئتك على مدار الساعة؛ بديهي تحاوره بلغة مفهومة ويحدّث شيفرتك، وكلما تمرنت على تجزئة المهام أنجزت أكثر في وقت أقصر. وهو رائع للتطوير التكراري: ابنِ قليلًا، شغّل ولاحظ، ثم اطلب ضبطًا وكرر.

Windsurf 1.4.4: بيئة تطوير تفهرس قاعدة الشيفرة كاملة

نجم صاعد آخر هو Windsurf، بيئة تأخذ فهم الشيفرة إلى مستوى أبعد. بناها فريق Codeium، وتتميز بفهرسة قاعدتك كلها واستعمال تقنيات استرجاع لتغذية النموذج بالقطع ذات الصلة. عمليًا تبرع في المشاريع الكبيرة حيث تتوزع إجابة سؤالك على ملفات كثيرة؛ جوهرها يستعمل «التوليد المعزز بالاسترجاع (RAG)»: يبحث عن أجزاء الشيفرة ذات الصلة بتوجيهك ويقدمها سياقًا، فتتوافق الاقتراحات مع شيفرتك القائمة.

كيف يبدو هذا؟ لنفرضك جديدًا على قاعدة كبيرة وتحتاج إضافة ميزة؛ تسأل:

```
Where in the codebase is the user authentication logic
handled?
```

فيبحث في الفهرس ويشير إلى الملف الصحيح بل الدالة. ثم تفتح محادثة (تسميها Windsurf واجهة «Cascade» وتستدعى بـ Cmd+L) وتقول:

```
Add a phone-based two-factor authentication to the login
flow.
```

ولأن Windsurf يملك سياق منطق التحقق، يولّد تغييرات ممتدة على ملفات متعددة — قاعدة بيانات وAPI وواجهة — بخيارات تتواءم مع بنية نظامك. ويستطيع وضع الكتابة (Write) تطبيق التغييرات بجرأة: ينشئ ملفات أو يعدّل قائمة تلقائيًا بدل اقتراح فروق جانبية؛ ترى مشروعك يتطور في مكانه، ويتصرف كمبتدئ مستقل ينفذ ميزات عبر القاعدة حين يثق. (فلسفة Cursor أكثر تحفظًا بطلب التأكيد، وإن كانت نسخته الاحترافية تملك تطبيقًا تلقائيًا.)

لاستثماره بفعالية افهم نقاط قوته. **أسئلة قاعدة الشيفرة:** استعلم بلغة مفهومة كأنها Stack Overflow مخصص لمشروعك؛ نعمة للمشاريع الإرثية حيث يستغرق العثور على تعريف ساعات، ويجيب في ثوانٍ. **اقتراحات بسياق شامل:** لأنه يطعم النموذج ملفات ذات صلة، يحسن معالجة «أعد هيكلة وحدة الدفع لاستعمال أداة السجلات الجديدة» إذ يعرف الاثنين. **أنماط تشغيل:** له أنماط متعددة — إكمال تلقائي، محادثة، أوامر، وCascade الأوسع سياقًا، ووضع الكتابة المنفذ؛ وأنت تقرر قدر الاستقلالية الممنوحة.

وللفرق يمكن دمجه في التطوير اليومي كما Cursor. وعند المفاضلة يفضّل بعضهم Windsurf لسرعته وجرأته ولملاءمته المشاريع الكبيرة بفضل الفهرسة؛ بينما تبدو واجهة Cursor أألف لمستخدمي VSCode. وليس الاختيار حتميًا — بعضهم يحتفظ بالاثنين، أو توحد الفرق على واحد. وخلاصة: Windsurf خيار ممتاز إن أردت مساعدًا «يقرأ التوثيق والشيفرة» قبل الكتابة؛ يقلص فرص دوال مهلوسة أو أسماء مخطئة لأنه يستعلم. أعطه تعليمات واضحة ودعه ينطلق في وضع الكتابة للمهام الكبيرة، واستعمله مضبوطًا أكثر للحساس، وراجع دائمًا تغييراته — خاصة في شيفرة حرجة. ذكي لكنه ليس معصومًا؛ وبحكمة يشبه بيئة فائقة الذكاء تعرف مشروعك كله وتنفذ أفكارًا عبره.

1.5 نماذج الذكاء الاصطناعي: مشهد توليد الشيفرة

تغير مشهد الترميز بالذكاء جذريًا؛ تتنافس نماذج قوية متعددة على انتباه المطوّرين، منها عائلات Claude وGemini وOpenAI. حيث كان نموذج واحد يهيمن، يقدم النظام اليوم خيارات غنية، لكل منها نقاط قوة تلائم سيناريوهات مختلفة.

1.5.1 فئات النماذج وفهمها

تندرج نماذج الترميز عمومًا في فئات بحسب نهجها وقوتها:

محسّنة للسرعة. تستجيب سريعًا وتصلح لإكمال لحظي وتكرار سريع؛ زمن أقل مقابل دقة أقل قليلًا في المعقد.

عميقة الاستدلال. تأخذ وقتًا أطول «لتفكر»، وتتفوق في التنقيح المعقد والقرارات المعمارية وحل المشكلات متعدد الخطوات.

متعددة الوسائط. تعالج نصًا وشيفرة وصورًا ورسومًا وفيديو؛ قيمة لفهم توثيق بصري وعناصر واجهات.

مفتوحة المصدر. يبرز DeepSeek بقوة تقارب المغلق بلا اشتراك أو تسجيل، وإن نقصت ميزات كتوليد الصور وتصفح الويب.

1.5.2 اختيار النموذج المناسب لمهمتك

بدل البحث عن «أفضل» نموذج، يوائم المطورون الناجحون النموذجَ مع المهمة:

- للنمذجة السريعة والترميز العام تصلح نماذج سريعة واسعة الدعم اللغوي.
- للتنقيح المعقد وتصميم الأنظمة تصلح نماذج الاستدلال العميق التي تتتبع المنطق بمنهجية.
- لقواعد الشيفرة الكبيرة اختر نماذج بنوافذ سياق واسعة تحيط بالمشروع كله.
- للفرق الحريصة على الميزانية تقدم النماذج مفتوحة المصدر قيمة ممتازة بلا اشتراكات.

وتدعم أدوات كثيرة نماذج متعددة — إصدارات OpenAI وClaude وGemini ونماذج احتكارية أخرى — فتتيح التبديل بحسب المهمة.

1.5.3 نصائح عملية تنفع مع أي نموذج

بغض النظر عن النموذج، تحسّن ممارسات معينة النتائج دائمًا. أولًا، قدّم سياقًا غنيًا: لا تطلب «دالة معالجة دفع» فحسب، شارك نماذج بياناتك وأنماط شيفرتك ونهج معالجة الأخطاء ومتطلباتك — كلما زاد السياق تواءم الناتج مع قاعدتك.

وتتفوق النماذج الحديثة في مراجعة ناتجها: بعد تلقي شيفرة مولدة اطلب فحص مشكلات أو اقتراح تحسينات أو شرح منطق؛ يلتقط هذا النقد الذاتي عللًا خفية.

واستعمل قدرة النموذج على حفظ سياق المحادثة: ابدأ بتنفيذ أساسي ثم صقله بطلبات متابعة؛ هذا النهج التكراري خير من محاولة تحديد كل شيء مسبقًا.

ولكل نموذج فروق دقيقة: بعضها أشرح وبعضها أوجز؛ وبعضها يميل إلى نحو أحدث وآخر يلعب بأمان. تعلم هذه الميول يساعدك على صياغة توجيهات أفضل.

1.6 النماذج الكبرى

يتطور المشهد شهريًا؛ نماذج جديدة تتحدى القادة بانتظام، وصارت المنافسة حادة حتى استفاد المطوّرون من خيار وقدرات غير مسبوقة. الأهم ليس اختيار «المثالي» بل فهم كيفية استثمار قوة ما هو متاح. وتعتمد فرق كثيرة نهج المحفظة: نماذج سريعة للروتيني، وقوية للمعقد، ومتخصصة لمجالات معينة. وبعض البيئات يتيح التبديل السلس بين النماذج أثناء المهمة. والنجاح يأتي من فهم الخيارات وتطبيقها استراتيجيًا لتسريع سير عملك.

1.6.1 Google Gemini: القوة متعددة الوسائط

تمثل عائلة Gemini تحولًا جوهريًا بقدرات متعددة الوسائط أصلًا. بخلاف نماذج تدربت أساسًا على نص وشيفرة، صُمم Gemini ليفهم ويعمل عبر نص وشيفرة وصور وفيديو؛ ما يجعله قويًا لسير عمل يهم فيها السياق البصري كما النصي.

وتثبت هذه الطبيعة قيمتها خاصة في تطوير الويب: يشارك المطورون لقطات تصاميم، ويولّد Gemini تنفيذات مطابقة للأسلوب البصري؛ ويتفوق في فهم الرسوم وعناصر الواجهات، شريكًا مثاليًا لترجمة تصاميم بصرية إلى شيفرة تعمل. وتتجاوز قدرته تعرف الصور: يستدل على عناصر بصرية ويفهم أنماط تصميم ويحافظ على اتساق جمالي عبر مشروع كامل. ويتيح تكامله مع محررات شائعة وإضافات مثل Cline وCode Assist خيارات

تخصيص من تفضيل فردي إلى معيار فريق: أوامر لمهام متكررة، وقواعد تسري على كل توليد، وأنماط متسقة عبر قواعد كبيرة. وطبقته المجانية السخية تضعه في منال الطلاب والهواة والناشئين، بينما تخدم ميزاته المؤسساتية متطلبات معقدة.

وما يميز Gemini قدرته على التفكير بعمق مع سرعة عملية: يتناوب بين استجابات سريعة للمهام البسيطة واستدلال ممتد للمعقدة، مكيّفًا مقاربته حسب المشكلة؛ وهذه المرونة مع فهمه البصري تجعله فعالًا خاصة في تطوير كامل الطبقات حيث منطق الخادم وجمال الواجهة مهمان بالقدر نفسه.

1.6.2 Claude: بارع الاستدلال

يتمحور نهج Claude من Anthropic حول الشفافية والاستدلال العميق. ترسخت عائلة Claude، وخاصة Sonnet، قادرة على مهام هندسة معقدة تتطلب تحليلًا دقيقًا وحلًا خطوة خطوة. وما يميز Claude إظهاره عملية تفكيره، فيتابع المطور استدلاله ويتحقق من منطقه قبل التنفيذ.

وتمثل ميزة Artifacts تحولًا في تفاعل المطوّرين مع المساعدين: بدل تقديم الشيفرة في محادثة فحسب، ينشئ Claude مساحة عمل تُعرض فيها الشيفرة وتُحرر وتُعاين لحظيًا، بيئة قوية خاصة لتطوير الواجهات وتصور البيانات وكل سيناريو يسرّعه feedback بصري — يكرر المصممون ويختبرون ويهذبون في المحادثة نفسها.

ويظهر Claude أداء استثنائيًا في مقاييس هندسة واقعية، مترسخًا بين الأعلى في إصلاح العلل وتنفيذ الميزات وإعادة الهيكلة. وقوته ليست في التوليد فحسب بل في فهم السياق الأوسع: يحلل قواعد قائمة ويحدد أنماطًا وأنماطًا مضادة ويقترح تحسينات معمارية ويحافظ على اتساق مع أساليب راسخة؛ ما يجعله لا يقدر بثمن لمشاريع جديدة وصيانة إرث معًا. ويتيح نهجه في إدارة الذاكرة والسياق بناء فهم عبر جلسات ممتدة؛ ففي المشاريع الكبيرة يستخرج بنية القاعدة وقرارات التصميم وأنماطها ويحتفظ بها، فتصير اقتراحاته أوثق صلة كلما تقدم التطوير — أشبه بعضو فريق يألف المشروع مع الوقت منه بمساعد بلا حالة.

1.6.3 ChatGPT: الرفيق متعدد الاستخدامات

ترسخ ChatGPT أداة متعددة الاستعمالات، لا يقدَّر لميزة متخصصة بل لتعدد استخداماته وسعة معرفته. موقعه فريد: بينما تندمج نماذج في بيئات التطوير أو تقدم بيئات متخصصة، يخدم ChatGPT مستشارَ برمجة متاحًا دائمًا يبقيه المطور مفتوحًا في متصفحه طوال اليوم.

وتجعل واجهته المحادثية منه فعالًا في حل المشكلات الاستكشافي والتعلم؛ يستعمله المطورون في «تنقيح البطة المطاطية» — يلصقون شيفرة مشكلة ويفكرون بصوت مسموع. ويدرك أنماطًا عبر كل لغة وإطار شائعين؛ سواء تنقيح تعبير نمطي، أو فهم رسالة خطأ غامضة، أو استكشاف توثيق غير مألوف، يقدم رؤى من معرفته الشاملة.

وقوته في جسر الهوة بين قصد الإنسان وتنفيذ الشيفرة؛ يتفوق في الترجمة بالاتجاهين: يحوّل وصفًا بلغة طبيعية إلى شيفرة تعمل، ويشرح شيفرة معقدة بلغة مفهومة؛ ما يجعله لا يقدر بثمن للتوثيق ومراجعات الشيفرة ونقل المعرفة. يمتد تعدده وراء لغات البرمجة إلى ملفات إعداد وسكربتات وصيغ بيانات؛ وبينما تتفوق أدوات متخصصة في نطاقاتها، يقدم ChatGPT مساعدة عبر طيف المهام كله — اتساع مفيد خاصة عند العمل على حدود تقنيات مختلفة، وقدرته على حفظ السياق عبر محادثات ممتدة تتيح استكشاف مشكلات معقدة وصقل الحلول بحوار.

1.7 اختيار النموذج المناسب لاحتياجاتك

يمثل توافر هذه المساعدات تحولًا جوهريًا في ممارسات التطوير. بدل رؤيتها خيارات متنافسة، يدرك الناجحون أن كل عائلة تجلب قوى فريدة لجوانب مختلفة: يتفوق Gemini حين يكون السياق البصري حاسمًا، خاصة في الواجهات ومع مواصفات التصميم؛ ويتألق Claude في استدلال عميق وإعادة هيكلة معقدة ونهج شفاف؛ وتوفر عائلة OpenAI تعدد استخدامات ومعرفة واسعة مثالية للتعلم والتنقيح والتحديات العابرة للنطاقات.

وتستعمل فرق كثيرة نهج المحفظة — نماذج مختلفة لمهام مختلفة داخل المشروع نفسه. سير عمل نموذجي قد يستعمل Gemini لترجمة نماذج تصميم إلى تنفيذات أولية، وClaude لقرارات معمارية ومراجعات، وChatGPT لحل عام وتوثيق. هذا النهج يعظم الإنتاجية بمواءمة قوة كل أداة مع تحديها. ومع استمرار التطور، يكمن المفتاح لا في اختيار «أفضل» واحد بل في فهم كيفية تنسيق مساعدين متعددين لتسريع كل جانب من دورة التطوير وتعزيزه.

هذا النظام البيئي فتي ويتغير بسرعة؛ لاعبون وقدرات جديدة تظهر كل بضعة أشهر. والخلاصة أنك لست مضطرًا لبناء ذكائك من الصفر لتستفيد من البرمجة بالقصد — ثمة أدوات كثيرة تجلب هذه القوة إلى أطراف أناملك، وسيناقش المؤلف منصات مختلفة وملاءمتها لسير عمل «فايب كودينغ» عبر الكتاب.

1.8 فوائد «فايب كودينغ» وحدوده: نظرة متوازنة

من المهم تعرف السيناريوهات التي يتألق فيها التطوير المُعان — وتلك التي قد يخفق فيها. لنستكشف حالات استعمال مثالية يتفوق فيها «فايب كودينغ»، ومواقف ما يزال الذكاء يتعثر فيها أو يتطلب تدخلًا بشريًا كثيفًا.

1.8.1 الحالات المثالية لنهج «فايب كودينغ»

كما تلائم معماريات معينة مشكلات معينة، لـ«فايب كودينغ» نقاط قوة في مشهد التطوير:

تطوير منتج من الصفر إلى الواحد. «فايب كودينغ» يغيّر قاعدة اللعبة لإطلاق مشروع جديد كليًا؛ «من الصفر إلى الواحد» (الذي نشره بيتر ثيل) يعني خلق شيء جديد من لا شيء. مع الذكاء تنتقل من لوحة فارغة إلى نموذج أولي يعمل بسرعة البرق. تحتاج تطبيق ويب لم يوجد قط؟ تولّد شيفرة نمطية لواجهتك وخادمك ومخطط قاعدتك وسكربتات نشرك في جلسة توجيه واحدة؛ مثالي لناشئين وهاكاثون يريدون التحقق من فكرة سريعًا. بدل أسابيع في نصب سقالة المشروع يفعلها الذكاء في دقائق. وقد روى مطورون بناء MVP في عطلة نهاية أسبوع بمساعدة ذكية — شيء كان يستغرق شهرًا فردًا. وبإخراج الفكرة سريعًا إلى منتج تختبرها مع مستخدمين أبكر. الذكاء عظيم في العام — توجيه، ومكونات واجهة أساسية، وعمليات CRUD — فيتفرغ أنت للجديد في منتجك.

غير أنه متى كسب MVPك زخمًا واتجه للإنتاج، يجب أن يتحول نهجك: هنا تصير الهندسة المُعانة ضرورية. فبينما ساعدك فايب كودينغ على الاستكشاف والتحقق سريعًا، يتطلب التوسع ممارسات أعمد: إعادة هيكلة بمعالجة أخطاء لائقة، وتغطية اختبارات شاملة، وحدود معمارية واضحة. الانتقال من نموذج إلى منتج هو التطور الطبيعي من حرية الاستكشاف إلى انضباط الهندسة؛ الفرق الذكية تدرك نقطة الانعطاف وتعدل استعمالها للذكاء — تحافظ على الزخم وتدخل حواجز الحماية اللازمة لنمو مستدام.

نمذجة الميزات وتطبيقات CRUD. كثير من هندسة البرمجيات، خاصة تطبيقات الأعمال، يتضمن وظائف CRUD — إنشاء وقراءة وتحديث وحذف — حول بيانات. عمل صيغي يجيده الذكاء لأنه رأى أمثلة لا تحصى. احتجت إضافة وحدة «مخزون» بشاشات CRUD وواجهات؟ سيعالجها فايب كودينغ جيدًا: هجرات قواعد بيانات، نماذج ORM، نقاط نهاية API، ونماذج واجهات بتحقق — كامل الطبقات غالبًا بلا أخطاء لأن هذه الأنماط شائعة في تدريبه. وحتى إن كانت لتطبيقك قواعد مخصصة تحددها في التوجيه وتحصل على مسودة لائقة. النتيجة: ما كان مهمة أسبوع من توصيل ممل تصير ظهيرة توجيه واختبار. ولأدوات داخلية ولوحات إدارة — وهي جوهريًا تطبيقات CRUD كبيرة — قد تعتمد على الذكاء تقريبًا كليًا.

وتصير الهندسة المُعانة حاسمة حين تتضمن عمليات CRUD منطق أعمال معقدًا أو قواعد تحقق أو تكاملًا مع أنظمة قائمة. يولّد فايب كودينغ الهيكل سريعًا، وتضمن الهندسة المُعانة معالجة حالات حدّية كتحديثات متزامنة والحفاظ على سلامة مرجعية واتباع أنماط مؤسستك. مثلًا تستعمل فايب كودينغ لسقالة CRUD الأولية، ثم تتحول لوضع المهندس لتنفيذ قواعد نطاقية كتنبيهات عتبات المخزون وتكامل مع أنظمة تحقق قائمة. المفتاح معرفة متى تنتقل من التوليد السريع إلى التهذيب الحريص.

شيفرة لصق وتكامل. تحتاج دمج خدمتين أو واجهتين؟ ذلك يتضمن غالبًا قراءة توثيق وكتابة شيفرة تحوّل بيانات من صيغة لأخرى. تدربت النماذج على توثيق واجهات وأمثلة شيفرة، فتسرّع التكامل: اطلب من ChatGPT أن يريك استدعاء واجهة الخدمة A من اللغة B — غالبًا ينتج شيفرة مثال بنقاط النهاية الصحيحة وربما مثال تحقق. دمج أنظمة متعددة — بوابة دفع مع نظام طلباتك، أو SDK تحليلات — يصير أسهل حين يقترح الذكاء الشيفرة النمطية وحالات حدّية. يتفوق في أنماط التكامل القياسية.

استغلال الأطر الحديثة. مساعدات الترميز «قرأت الأدلة» لكل الأطر الشائعة: React وAngular وDjango وRails وNode/Express وFlutter. فإن استعملت إطارًا معروفًا يولّد الذكاء شيفرة اصطلاحية له: مكوّن React بخطافات وإدارة حالة، أو نموذج Django بفئة إدارة وserializer ملائمين. الفائدة ألا تتذكر كل تفصيلة — الذكاء يملأ الفجوات. ويؤدي فايب كودينغ جيدًا خاصة في مهام ويب حديثة كتوليد HTML/JSX بالفئات الصحيحة أو توصيل نقاط نهاية، لأنها مهام رآها مرارًا — تكبير أطر يجلس جانبك يكتب النمطي بينما تقرر أنت خصوصيات الميزة.

توليد شيفرة تكرارية. أحيانًا يلزم إنشاء كثير من شيفرة متشابهة — نقاط نهاية أو أصناف متماثلة لكل نوع في مخطط — عمل ممل وعرضة للخطأ على الإنسان. أما الذكاء فيحب البنى التكرارية: أرِه مثالًا أو اثنين فينتج الباقي باتساق، قد يوفر توليدًا بالجملة وقتًا هائلًا: إن كتبت أصناف نماذج بيانات لخمسين نوع سجل، وجّه بمثال واحد واطلب توليد أصناف الأنواع الخمسين على النمط؛ غالبًا يفعل بلا عيب وفي ثوانٍ، فتتجنب يومًا كاملًا من ترميز رتيب.

حين تتقدم الهندسة المُعانة. بينما يتفوق فايب كودينغ في سيناريوهات، تصير الهندسة المُعانة لا غنى عنها في أخرى؛ فهمها يساعد على اختيار النهج الصحيح من البداية، متجنبًا إعادة كتابة مكلفة وديونًا تقنيًا. **التنفيذات الخوارزمية المعقدة** تتطلب هندسة: بنى بيانات متطورة أو خوارزميات حرجة الأداء أو مسائل جديدة تحتاج سيطرة دقيقة؛ هنا يخدم الذكاء مساعدًا عليمًا لا مولدًا — يشرح مقاربات ويراجع تنفيذك، لكنك تمسك قرارات المعمارية والتحسين. **الأنظمة الحرجة** تطلب صرامة منذ البدء: معاملات مالية ورعاية صحية وبنى أمنية لا تحتمل استكشافًا؛ كل سطر يحتاج نظرًا دقيقًا واختبارًا شاملًا وامتثالًا غالبًا؛ يساعد الذكاء بأفضل ممارسات وتحديد ثغرات، لكن يبقى للمطور سيطرة محكمة؛ كلفة الفشل تفوق أي ميزة سرعة. **تكامل أنظمة إرثية** يقدم تحديات يثبت فيها الانضباط جوهره: قواعد عمرها عقود وبروتوكولات احتكارية ودين تقني هائل — غالبًا يفشل مطابقة أنماط فايب كودينغ؛ يلزم فهم عميق لقيود قائمة وتخطيط حريص وإعادة هيكلة منهجية؛ يفسر الذكاء أنماطًا إرثية ويقترح تحديثًا، لكن التنفيذ يحتاج دقة لا توفرها إلا هندسة منظمة. **وتحسين الأداء** مجال تتفوق فيه الهندسة: نادرًا ما ينتج الذكاء حلولًا مثلى لمسارات حرجة؛ إدارة ذاكرة وتحسين مخابئ ومعالجة توازية وخفض كمون تتطلب فهمًا عميقًا للعتاد ونظم التشغيل وتعقيد الخوارزميات؛ هنا يخدم الذكاء مساعدَ بحث يستكشف تقنيات ويقارن مقاربات، بينما تتخذ أنت القرارات المستنيرة.

في هذه السيناريوهات تتواءم قدرة الذكاء على التعرف على الأنماط وسرعته مع المهمة. جوهريًا، يزدهر فايب كودينغ في مهام أراضيها مطروقة — CRUD وبنى ويب نموذجية — ومهام تستفيد من تجربة وخطأ سريع — نماذج أولية وأفكار — كمبتدئ قرأ كل مستودعات GitHub ويستحضر فورًا كيف يُفعل عادة ويكتبه لك لتراجعه. ذلك قوي جدًا لتحريك الأمور سريعًا.

التعرف على نقاط التحول. يكمن فن التطوير المعزز لا في اختيار نهج على آخر بل في معرفة متى تنتقل بينهما؛ المطورون الناجحون يطورون حدسًا لنقاط الانعطاف: تبدأ ميزة جديدة؟ ابدأ بفايب كودينغ لاستكشاف سريع. تلاحظ الشيفرة تصير معقدة أو تلمس أنظمة حرجة؟ تحول لوضع المهندس. تبني إثبات مفهوم لعرض عميل؟ فايب كودينغ يوصلك سريعًا. تحوّله نظام إنتاج؟ حان وقت انضباط الهندسة. هذه السيولة — التنقل بسلاسة بين استكشاف سريع وبناء حريص — هي ما يميز مطورين معززين فعالين؛ يفهمون أن النهجين أداتان متكاملتان في عدتهم تلائم كل منهما مراحل مختلفة من دورة التطوير. الهدف ليس اختيار طرف، بل استثمار النهجين استراتيجيًا لتعظيم السرعة والجودة معًا.

1.8.2 حيث ما يزال الذكاء الاصطناعي يتعثر

على إعجاب الأدوات الحالية، ليست سحرًا. ثمة فئات مشكلات تبقى صعبة على الذكاء بموثوقية، وتتطلب غالبًا بصيرة بشرية أو تقنيات تقليدية؛ معرفتها تضبط التوقعات وتخطط متى تستند ومتى تستلم الزمام. تشمل الحدود ما يلي:

أنظمة معقدة بعمق. إن تعاملت مع خوارزميات شديدة التعقيد أو مشكلات جديدة لم يرها الذكاء، اضطرب؛ كتابة خوارزمية من ورقة بحث أو بناء مصرف أو نظام شديد التوازي — منطق متشابك يتطلب فهمًا حقيقيًا وقفزات إبداعية. قد يحاول الذكاء لكنه يخطئ بخطأ خفي. وفي نطاقات كهذه تؤدي ميوله إلى شيفرة «صحيحة تقريبًا لا بدقة» إلى أخذ ورد كثير؛ وكما يناقش الفصلان الثالث والرابع، الـ 30% الأخيرة من الصحة عسيرة على الذكاء — «مشكلة السبعين بالمئة»: يوصلك معظم الطريق سريعًا لكن الجزء الأخير عسير. مطوّر خبير يستعمل الذكاء لمياكل ودوال مساعدة لكنه يكتب المنطق الجوهري بنفسه.

تحسينات منخفضة المستوى وبرمجة أنظمة. دربت النماذج أساسًا على لغات وتجريدات عالية؛ احتجت تلاعبًا بالبتات أو شيفرة C محسنة لميكرومتحكم أو تعليمات SIMD متجهة، فقد لا يكون الذكاء موثوقًا؛ قد ينتج شيفرة تبدو معقولة لكنها ليست مثلى بل ولا صحيحة على مستوى العتاد. وبالمثل لإدارة ذاكرة وقيود زمن حقيقي لا يملك الذكاء مفهومًا حقيقيًا لها؛ فلشيفرة حرجة الأداء تختبر اقتراحاته جيدًا أو تكتبها يدويًا. مع ذلك قد يساعد بقالب بداية أو شرح تجميع، لكن لا تأتمنه أعمى هنا.

أطر فريدة أو نادرة. إن استعملت إطارًا جديدًا جدًا أو غامضًا لم يكن موجودًا أثناء تدريب الذكاء فلن يعرفه؛ قد يعمّم أو ينتج شيفرة تبدو ملائمة لكنها تستدعي دوالًا غير موجودة (هلوسات) أو تستعمل إصدارات واجهة قديمة. صدر إصدار جديد لإطار ويب الشهر الماضي بتغييرات كاسرة؟ لن يعرف؛ قد يعطيك شيفرة للإصدار القديم؛ تعود إلى التوثيق وربما «تدرّب الذكاء على الطاير» بإطعامه سياقًا منه داخل توجيهك.

تصميم واجهات وتجربة إبداعي. طلبت واجهة أو تجربة جديدة كليًا؟ ليس بارعًا في القفزة الإبداعية؛ يولّد شيفرة واجهة لأنماط معروفة — نموذج قياسي أو لوحة — لكن واجهة مبتكرة لا سوابق لها قد لا تلهمه — قد يرقع مكونات مألوفة. ما زال المصممون ومطورو الواجهات مطلوبين لحلم تجارب جديدة؛ بلغة الترميز: يصنع واجهة

قياسية المظهر سريعًا، لكن للإحساس المخصص الخاص توجّهه أو تهذبه يدويًا.

تأويل القصد والمتطلبات. يتعثر الذكاء أحيانًا حين تكون المتطلبات ضمنية أو متناقضة؛ لا فهم حقيقي لديه للهدف وراء ما تخبره به صراحة. غمضت المتطلبات («اجعله كفؤًا» — ماذا يعني بدقة؟) قد يخمن خطأ ما يهمك — ذاكرة أم سرعة. البشر أفضل في توضيح القصد، خاصة مع أصحاب مصلحة غير تقنيين؛ وقد يسيء تأويل تعليمات بسياق نطاقي لا يعلمه — كقواعد أعمال؛ فينتج حلًا صحيحًا منطقيًا لكنه لا يحل المشكلة الحقيقية لأن الفارق ضاع.

ومثال يجمع هذه العناصر: تخيّل تطوير محرك رسوميات ثلاثية جديد — نظام معقد — بلغة Rust — مستوى نظام حرج الأداء — ولديك خوارزميات جديدة للرسم — مسائل فريدة. قد يساعد الذكاء ببعض الشيفرة النمطية لكنك تعتمد جوهريًا على براعة بشرية للجوهر؛ قد يبدأ لك بنافذة وحلقة رسم أساسية — مهام شائعة — لكن للأجزاء المخصصة تتقدم بترميز حريص مع مساعدة خوارزمية بصيغة pseudocode. وإن طلبت تحسين حلقة ساخنة بلغة تجميع فسيلزمك التحقق من كل تعليمة. ويفتقر الذكاء أيضًا إلى بصيرة حل مشكلات؛ هو في النهاية مطابقة أنماط؛ تطلبت مشكلتك لمحة «آها!» فقد يتخبط مقدمًا أشياء تشبه الشيفرة ولا تحلها؛ هنا ينقذ الموقف إنسان يتراجع خطوة ويفكر تجريديًا أو يستند إلى خبرة — ومتى حصلت على البصيرة تستعمل الذكاء لتنفيذها سريعًا.

فهم هذه القوى والضعف يضمن نشر تقنيات فايب كودينغ في المواضع الصحيحة. لتعظيم النجاح استفد من الذكاء فيما يجيده — الأنماط المعروفة — وطبق إبداعك على الفريد من تطبيقك؛ وكن مستعدًا للتدخل في مناطق تعثره — مثلًا راجع بعناية أي شيفرة حساسة أمنيًا يكتبها، فقد يفوته حالة حدّية. استعمل الذكاء مكملًا لقوى الإنسان: دعه يتولى العرض — شيفرة كثيرة ونمطي — وتولَّ أنت العمق — منطق معقد ومعمارية؛ استعمله معززًا حيث يتفوق، ولا تخشَ إمساك المقود في المقاطع الأصعب — هذا يلعب على قوى الطرفين ويعطي أفضل نتيجة. معرفة متى تستعمل الذكاء ومتى تعتمد مهارة بشرية هي ما يجعلك مطورًا عالي الفعالية في هذا العصر.

كل تقنية جديدة تأتي بمزايا ومحاذير؛ ومع احتضاننا دفعة الإنتاجية والإبداع من التطوير المُعان، من المهم مقاربة الأمر بفهم دقيق للحدود والمقايضات. أبرز الفوائد:

دورات تطوير أسرع. تنتقل المشاريع من مفهوم إلى نموذج إلى منتج أسرع؛ يولّد الذكاء شيفرة سقالة في لمح البصر فتقضي وقتًا أكثر في الأجزاء الفريدة.

نمذجة وتجريب معززان. بكلفة تجربة أقل — صف ما تريد واحصل على مسودة سريعة — يشعر المطورون بحرية أكبر في التجريب؛ قد تنمذج مقاربات متعددة بتوجيه الذكاء بطرق مختلفة ثم تختار الأفضل؛ هذا التبادل التكراري قد يقود إلى حلول أكثر إبداعًا.

معرفة في أطراف أناملك. دربت النماذج على جسم هائل من المعرفة البرمجية؛ «تعرف» غالبًا واجهات غامضة أو حلول رسائل خطأ؛ عمليًا قد تظهر حلولًا أو أفكارًا لم تفكر فيها، جاعلة إياك حلال مشكلات أكثر فاعلية.

اتساق وتقييس. في بيئات الفرق قد يساعد مساعد ذكي في فرض معايير ترميز وأفضل ممارسات بتوليد شيفرة بأسلوب متسق؛ وإن هُيئ بدليل أسلوب مشروعك ضمن اتباع الجميع أنماطًا متشابهة؛ وحتى بدون تدريب صريح تنتج النماذج غالبًا شيفرة اصطلاحية — إذ تعلمت من ملايين الأمثلة؛ ما قد يقلل جهد مراجعات الشيفرة إذ تبدو دواله مألوفة وملتزمة اصطلاحات شائعة.

ومن الحدود والمقايضات التي ينبغي اعتبارها:

جودة ناتج متقلبة. ليست النماذج معصومة؛ قد تنتج شيفرة تبدو صحيحة وبها علل خفية أو عدم كفاءة؛ وقد تختار نهجًا قديمًا لأن بيانات تدريبها ضمت شيفرة أقدم. على المطور البقاء متيقظًا؛ فكما لا تلصق شيفرة من الإنترنت بلا فهم، لا تقبل شيفرة ذكاء بلا تفكير؛ سيناقش الجزء الثاني تقنيات للتحقق من الشيفرة المولدة واختبارها بدقة.

غموض التوجيه يقود إلى غموض الشيفرة. إن كان توجيهك ناقص التحديد فعلى الذكاء تخمين قصدك — وقد يخطئ. قلت «رتب قائمة أسماء» فقد يفترض ترتيبًا أبجديًا، وربما قصدت ترتيبًا بطول الاسم؛ لن يعرف الفرق ما لم توضح. لهذا التحديدُ في التوجيهات حيوي (موضوع الفصل الثاني)؛ ستتعلم توقع التفاصيل التي يلزم تهجئتها.

اعتماد مفرط وضمور مهارات. إن اعتمد مطورون جدد دائمًا على الذكاء في كتابة شيفرتهم، أيطورون العمق نفسه في فهم الخوارزميات والتنقيح؟ ثمة خطر ضمور مهارات، كما قد يضعف الاعتماد على GPS حسّ الاتجاه. للتخفيف استعمل الذكاء أداة تعلم — انتبه للشيفرة واسأل لماذا — وتمرّن أحيانًا على الترميز دونه لتحتفظ بأساسياتك.

مخاوف خصوصية وأمان. استعمال أدوات ذكاء سحابية يعني غالبًا إرسال شيفرتك — التي قد تكون احتكارية أو حساسة — إلى خدمة طرف ثالث للتحليل؛ على الشركات اعتبار ذلك. تعالجه أدوات كثيرة بنماذج محلية أو ضمانات بعدم حفظ الشيفرة، لكنه يبقى اعتبارًا. وثمة خطر أن يولّد الذكاء مصادفة شيفرة شديدة الشبه بشيء في بيانات تدريبه قد يكون برخصة مفتوحة كـ GPL؛ احتمال ضئيل — وثمة تدابير لمنع مخرجات حرفية طويلة — لكنه يبرز الحاجة إلى مراجعة ناتج الذكاء وفهمه قبل دمجه. يغوص الفصل الثامن في الأمان والموثوقية.

انحياز في ناتج الذكاء. قد تعكس النماذج انحيازات بيانات تدريبها؛ في الترميز قد يكون حميدًا كتفضيل أسماء متغيرات معينة، أو جوهريًا كاستعمال أمثلة تفترض صفات مستخدمين محددة؛ مثلًا قد يستعمل foo/bar لكل متغيرات أمثلته — لأن أمثلة كثيرة فعلت — أو يفترض أمورًا عن لغات مستخدمي منطقته. غالبًا ليست مشكلة كبيرة في توليد الشيفرة مقارنة بتطبيقات ذكاء أخرى، لكنه وعي يستحق. وأدق أن يميل الذكاء إلى حلول رآها أكثر حتى إن لم تكن الأفضل لحالتك. يناقش الفصل التاسع الانحياز واعتبارات أخلاقية أخرى.

عوامل بشرية وثقة. ليس كل المطوّرين مرتاحين فورًا لهذا الأسلوب؛ للترميز متعة وفن معينان وقد يشعر بعضهم أن مشاركة الذكاء تنقصهما؛ وقد يكون ثمة انعدام ثقة أولي — «أحقًا فعل هذا صحيحًا؟» — لا تتجاوزه إلا ممارسات جيدة ووقت. على الفرق المعتمدة للذكاء السماح بفترة تكيف وتشجيع مشاركة الخبرات والنصائح؛ ومع الوقت، كما مع أي أداة، يجد معظمهم توازنًا تقدر فيه مساهمات الذكاء ويركز الخبراء على ما يحسنه البشر أفضل.

1.9 الخلاصة والخطوات التالية

يقدم التحول نحو البرمجة بالقصد إمكانات هائلة لجعل تطوير البرمجيات أسرع وأيسر منالًا وبوجوه كثيرة أمتع. لكن تحقيقها يعني فهم الديناميكيات الجديدة: كيف نتواصل مع الذكاء بفعالية، وكيف نتحقق من ناتجه، وكيف ندمجه بمسؤولية في عملية تطويرك.

منظور المؤلف — المصقول من العمل بهذه الأدوات ومراقبة مشاريع كثيرة — أن أفضل استعمال للذكاء يكمن في دمج «الانطباع» الإبداعي مع نظافة هندسية رصينة. شجع الأفكار الجامحة والمسودات السريعة التي يقدمها الذكاء — تلك قوى خارقة جديدة — لكن وجّهها بحكمة راكمتها البرمجة عبر عقود: أهمية التخطيط والاختبار وفهم ما تبني.

حين نضرب ذلك التوازن نحوز أفضل العالمين: برمجيات تبنى أسرع وربما بخيال أوسع، وبرمجيات نثق بها ونصونها ونمنيها بثقة. ذلك، في المحصلة، هو كيف نرتقي بحرفتنا في عصر الذكاء: لا باختيار الانطباع على الهندسة أو العكس، بل بإتقان الطيف كله بينهما.

إضافة تعليمية: أسئلة للمراجعة والنقاش:

1. ما الفرق الجوهري بين «فايب كودينغ» والهندسة المُعانة من حيث الهدف والتوقعات؟
2. لماذا تساعد أدوات الذكاء الخبير أكثر من المبتدئ في رأي المؤلف؟
3. ما «مفارقة المعرفة»؟ أعطِ مثالًا عليها من تجربتك أو قراءتك.
4. عدّد ثلاث حالات يتفوق فيها فايب كودينغ، وثلاثًا تتقدم فيها الهندسة المُعانة.
5. ما دور «نافذة السياق» في عمل المساعدات الذكية على مشاريع كبيرة؟
6. لماذا لا يجوز نشر شيفرة كتبها الذكاء دون فهمها؟ بمن شبّه المؤلف الذكاء؟

تاليًا، يستكشف الفصل الثاني فن صياغة التوجيهات والتعاون مع الذكاء. ومع مفاهيم هذا الفصل التأسيسية صرت جاهزًا لاستكشاف الجانب العملي من عصر البرمجة الجديد — ما يرسي الأساس لأمثلة تطبيقية وتقنيات توجيه أعمق في الفصول التالية.

معجم الدفعة الأولى

> **إضافة تعليمية من الإعداد العربي.** هذه تعريفات تمهيدية مرتبطة بالمقدمة والفصل الأول، وليست تعريفات معيارية تستوعب كل الحالات. نلتزم بهذه المقابلات في الدفعات اللاحقة، ونوثّق أي تغيير اصطلاحي بدل تبديل الكلمات بصمت.

النهجان والمفاهيم المركزية

المصطلح الإنجليزي	المقابل العربي	المعنى في هذا الكتاب
Vibe coding	«فايب كودينغ»	نهج استكشافي يبدأ بالتوجيه: تصف ما تريد بلغة طبيعية وتدع النموذج يملأ الفراغات؛ نُبقي اللفظ منقولًا لأنه مصطلح حديث متداول، ونشرحه عند أول ظهور.
AI-assisted engineering	الهندسة المُعانة بالذكاء الاصطناعي	طريقة منضبطة تدخل الذكاء في كل مرحلة من التطوير ضمن خطة وقيود واضحة؛ «خطة أولًا» مقابل «توجيه أولًا».
Programming with intent	البرمجة بالقصد	التعبير عن الهدف (ماذا) بلغة عالية المستوى وترك الذكاء يستنتج التنفيذ (كيف).
Prompt	توجيه (برومبت)	الوصف أو السؤال الذي يُدخل إلى نظام ترميز ذكي؛ نلتزم «توجيه» بعد ذكر الأصل مرة.
Prompt engineering	هندسة التوجيه	مهارة صياغة توجيهات واضحة محددة للحصول على ناتج أدق.
Large language model / LLM	نموذج لغوي كبير	نموذج بمليارات البارامترات دُرِّب على نصوص وشيفرة هائلة ليولّد نصًا وشيفرة مرجحة.
Intent	القصد	ما يريد المطوّر إنجازه، معبَّرًا عنه عالي المستوى.
Context window	نافذة السياق	قدر النص/الشيفرة الذي يحيط به النموذج ويفهمه دفعة واحدة؛ أطولها يتيح فهم مشاريع كاملة.
Token	رمز	قطعة كلمة أو رمز شيفرة يولدها النموذج واحدة واحدة.
Hallucination	هلوسة	توليد النموذج دوالًا أو واجهات غير موجودة تبدو معقولة.
Boilerplate	شيفرة نمطية	شيفرة إعداد متكررة قليلة الخصوصية يبرع الذكاء في توليدها.
Codebase	قاعدة الشيفرة	كامل شيفرة المشروع وملفاته التي تفهرسها الأدوات الذكية.
Diff	فرق التغييرات	عرض التعديلات المقترحة لاعتمادها أو رفضها.

MVP	الحد الأدنى القابل للتطبيق	أول نسخة من منتج تحقق الفكرة وتُختبر بها.
Zero to one	من الصفر إلى الواحد	إنشاء شيء جديد كليًا لا تطوير قائم.
CRUD	عمليات CRUD	إنشاء وقراءة وتحديث وحذف؛ العمل الصيغي الشائع في تطبيقات الأعمال.
SDLC	دورة حياة تطوير البرمجيات	مراحل التطوير من التصميم إلى الاختبار والنشر.
AI pair programmer	مبرمج زوجي ذكي	تشبيه المساعد الذكي بشريك برمجة يجاورك.
Knowledge paradox	مفارقة المعرفة	الذكاء يسرّع الخبير أكثر من المبتدئ؛ الخبير يسرّع ما يعرفه والمبتدئ يحاول التعلم به.
The 70% problem	مشكلة السبعين بالمئة	الذكاء يقطع معظم الطريق سريعًا لكن الـ 30% الأخيرة من الصحة عسيرة.

الأدوات والبروتوكولات

المصطلح الإنجليزي	المقابل العربي	المعنى في هذا الكتاب
VSCode	VSCode	محرر الشيفرة الأشهر، منصة مايكروسوفت بعد تكامل Copilot. يُترك باللاتينية.
GitHub Copilot	GitHub Copilot	مساعد الترميز المدمج في VSCode: إكمال مضمّن، محادثة، ووضع وكيل.
Agent mode	وضع الوكيل	نمط يستدعي أدوات وينفذ خطوات ذاتيًا حتى اكتمال المهمة، بمراجعة بشرية.
MCP	بروتوكول سياق النموذج (MCP)	طريقة معيارية تكتشف بها النماذج أدوات وبيانات خارجية وتتفاعل معها.
Cline	Cline	إضافة مفتوحة المصدر تحول VSCode إلى وكيل ترميز مستقل (Claude Dev سابقًا).
Cursor	Cursor	محرر «ذكاء أولًا» (تفريعة VSCode) بمحادثة ووضع Composer.
Composer	وضع الملحن (Composer)	نمط في Cursor لتوليد شيفرة متعدد الخطوات.

Windsurf	Windsurf	بيئة تطوير بفهرسة كاملة لقاعدة الشيفرة وتوليد معزز بالاسترجاع.
Cascade	واجهة Cascade	المحادثة الفائقة في Windsurf ذات السياق الأوسع.
Write mode	وضع الكتابة	نمط يطبق التغييرات على الملفات مباشرة بدل اقتراح فروق.
RAG	التوليد المعزز بالاسترجاع (RAG)	استرجاع أجزاء الشيفرة ذات الصلة وتغذيتها للنموذج سياقًا.
Artifacts	ميزة Artifacts	مساحة عمل في Claude تُعرض فيها الشيفرة وتُحرر وتُعاين لحظيًا.
Ollama	Ollama	وسيلة تشغيل نماذج محلية تدعمها أدوات مثل Cline.

نماذج وعائلات

المصطلح الإنجليزي	المقابل العربي	المعنى في هذا الكتاب
Claude	Claude	عائلة نماذج Anthropic؛ تميزها الشفافية والاستدلال العميق. باللاتينية.
Gemini	Gemini	عائلة Google متعددة الوسائط أصلًا. باللاتينية.
ChatGPT	ChatGPT	مساعد OpenAI متعدد الاستخدامات؛ «سكين الجيش السويسري». باللاتينية.
DeepSeek	DeepSeek	بديل مفتوح بقوة تقارب المغلق بلا اشتراك. باللاتينية.
Speed-optimized models	نماذج محسّنة للسرعة	زمن استجابة أقل مقابل دقة أقل قليلًا في المعقد.
Deep-reasoning models	نماذج عميقة الاستدلال	أبطأ وأدق في التنقيح المعقد والقرارات المعمارية.
Multimodal models	نماذج متعددة الوسائط	تفهم نصًا وشيفرة وصورًا ورسومًا وفيديو.

ملاحظات الإعداد والمراجع

المصدر وحدود التغطية

المصدر هو **Beyond Vibe Coding: From Coder to AI-Era Developer**، الطبعة الأولى، تأليف **Addy Osmani**، نشر **O'Reilly Media** (أغسطس 2025). نعتمد ملف `BeyondVibeCoding.pdf` المرفق بالمستودع في `sources/beyond-vibe-coding-1st-en/`، وعدد صفحاته **255 صفحة** بترقيم فيزيائي يبدأ من 1. تُسجَّل بصمة الملف وبيانات نشره في `source.json` بجانبه.

لا تُشتق إحالات الصفحات هنا من العلامات المرجعية المضمنة في PDF وحدها؛ جرى التحقق من بدايات الفصول بمواضع عناوينها في نص الصفحات (الفصل الأول يبدأ فعلًا في الصفحة 17). يعرض `manifest.json` في مشروع الكتاب نطاقات المصدر وحالة كل فصل، بما فيها الفصول غير المعدّة بعد.

مادة هذه الدفعة	موضعها في ملف المصدر	طريقة التعامل معها
المقدمة العربية	مقدمة الأصل، الصفحات من 9 إلى 14	إعادة صياغة تعليمية للجمهور والتوقعات والاتفاقيات، مع إرشادات عربية إضافية موسومة.
الفصل الأول	الصفحات من 17 إلى 52	إعداد تعليمي لجميع أقسامه التسعة وأقسامه الفرعية الثمانية عشر، مع الحفاظ على ترتيبها وإعادة رسم أشكاله الثلاثة.

حدود التغطية

- لا تشمل هذه الدفعة الفصول من الثاني إلى الحادي عشر؛ تُذكر عناوينها في خريطة الإنجاز بصفتها **لم تُعد بعد**، لا بصفتها محتوى.
- صفحات افتتاح الأجزاء الثلاثة (15 و79 و155) مواد أصلية قائمة بذاتها، لم تُعد بعد وتُذكر في خريطة الإنجاز ضمن المواد الأخرى.
- الفهرس (239–253) ونبذة المؤلف والخاتمة الطباعية (254–255) لم تُعد بعد.
- أقسام المقدمة الأصلية الخاصة بناشر الكتاب (O'Reilly Online Learning) وطرق التواصل معه لم تُنقل بصفتها محتوى تعليميًا؛ هي بيانات ناشر لا مادة فصل.

مراجع وإحالات داعمة

1. صفحة الكتاب لدى الناشر وقائمة التصويبات: https://oreil.ly/BeyondVibeCoding. تُراجع التصويبات قبل اعتماد أي مثال حساس في دفعات لاحقة.

2. موقع O'Reilly: https://oreilly.com لبيانات الناشر والطبعة.
3. مصطلح «فايب كودينغ» صاغه Andrej Karpathy؛ يُذكر الأصل ذلك في المقدمة والفصل الأول، وتُنسب الاقتباسات المنقولة عنه إليه لا إلى المؤلف.
4. الأرقام والإحصاءات المنقولة (مثل نسب استطلاعات دمج الذكاء في سير العمل) وردت في الأصل مستندة إلى تدوينة المؤلف لـ Pragmatic Engineer (يناير 2025)؛ تُعامل أرقامًا منقولة عن المصدر لا نتائج تحقق مستقل في هذا الإعداد.

سجل التغطية وحدود الأمثلة

يسجل ملف التغطية `coverage/chapter-01.json` العناوين الإنجليزية والعربية وأرقام الصفحات لكل عنوان مرقم في الفصل الأول (سبعة وعشرون عنوانًا)، والأشكال الثلاثة. الأصل لا يطبع أرقامًا لأقسام الفصل الأول؛ اعتمدنا ترقيمًا تعليميًا متسلسلًا (1.1…1.9 وأقسامها الفرعية) موثقًا في حقل `numbering_note` داخل سجل التغطية. هذا سجل مقابلة، وليس بديلًا عن نص الفصل.

لا يتضمن الفصل الأول أوامر طرفية قابلة للتنفيذ؛ أمثلته توجيهات نصية ومقتبسات، عُرضت كتل التوجيه منها بصيغة `text` قابلة للنسخ. لذلك لا توجد مختبرات آلية في هذه الدفعة، ولا تُنفذ كتل نصية لمجرد ورودها في الأصل.

تبقى ملفات الدفعات السابقة لأي كتاب محفوظة كما سُلّمت؛ الدفعة الحالية أولية لهذا الكتاب ولا تعيد بناء مخرجات كتب أخرى. لم تُدّعَ مراجعة نهائية للمادة قبل وصول ملاحظات المستخدم.

خريطة إنجاز الكتاب

عدد الفصول ذات المسودات: 1 من أصل 11، إضافة إلى المقدمة العربية. هذه أعداد فصول، وليست نسبة مئوية من كلمات الكتاب أو صفحاته. «مسودة متاحة» تعني وجود نص قابل للقراءة والمراجعة، لا اعتماد ترجمة نهائية.

الفصل	العنوان العربي المقترح	صفحات ملف المصدر	الحالة
1	المقدمة: ما «فايب كودينغ»؟	17–52	مسودة متاحة
2	فن التوجيه: التواصل الفعال مع الذكاء الاصطناعي	53–78	لم يبدأ إعدادها
3	مشكلة السبعين بالمئة: سير عمل بمساعدة الذكاء تنجح فعلًا	81–94	لم يبدأ إعدادها
4	ما بعد السبعين بالمئة: تعظيم الإسهام البشري	95–110	لم يبدأ إعدادها
5	فهم الشيفرة المولّدة: راجع وصقّل وتملّك	111–122	لم يبدأ إعدادها
6	النمذجة الأولية بالذكاء الاصطناعي: أدوات وتقنيات	123–134	لم يبدأ إعدادها
7	بناء تطبيقات الويب بالذكاء الاصطناعي	135–154	لم يبدأ إعدادها
8	الأمان والقابلية للصيانة والموثوقية	157–184	لم يبدأ إعدادها
9	الأبعاد الأخلاقية لـ«فايب كودينغ»	185–200	لم يبدأ إعدادها
10	وكلاء الترميز المستقلون في الخلفية	201–218	لم يبدأ إعدادها

11	ما بعد توليد الشيفرة: مستقبل التطوير المعزز بالذكاء	219–238	لم يبدأ إعدادها

يفتتح الأصل أجزاءه الثلاثة بصفحات عنوان (15 و79 و155) لا تُعد فصولًا؛ تقع ضمن نطاقات الفصول المجاورة في الخريطة.

مواد أخرى لم تُعد بعد: صفحات افتتاح الجزء الأول، صفحات افتتاح الجزء الثاني، صفحات افتتاح الجزء الثالث، فهرس الأصل، نبذة عن المؤلف والخاتمة الطباعية.

الخطوة التالية بعد المراجعة: انتظار ملاحظات المستخدم على الدفعة الحالية، ثم إعداد الفصل 2: فن التوجيه: التواصل الفعال مع الذكاء الاصطناعي. لا يحتوي هذا الإصدار على صفحات فارغة أو ملخصات بديلة تدّعي أنها الفصول المتبقية.